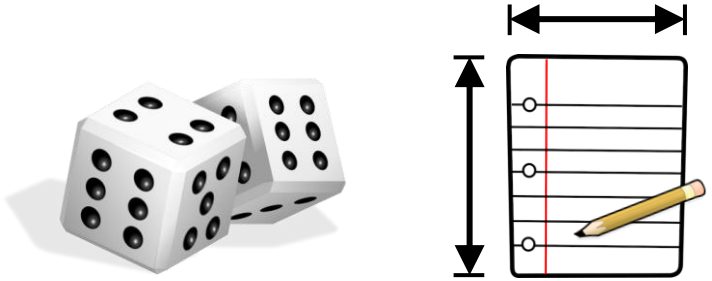
OpenAI's Proof That $\mathrm{L} = \mathrm{BPL}$



William Hoza

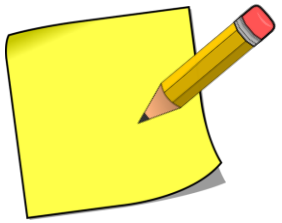
The University of Chicago

October 26, 2026

The complexity class BPL



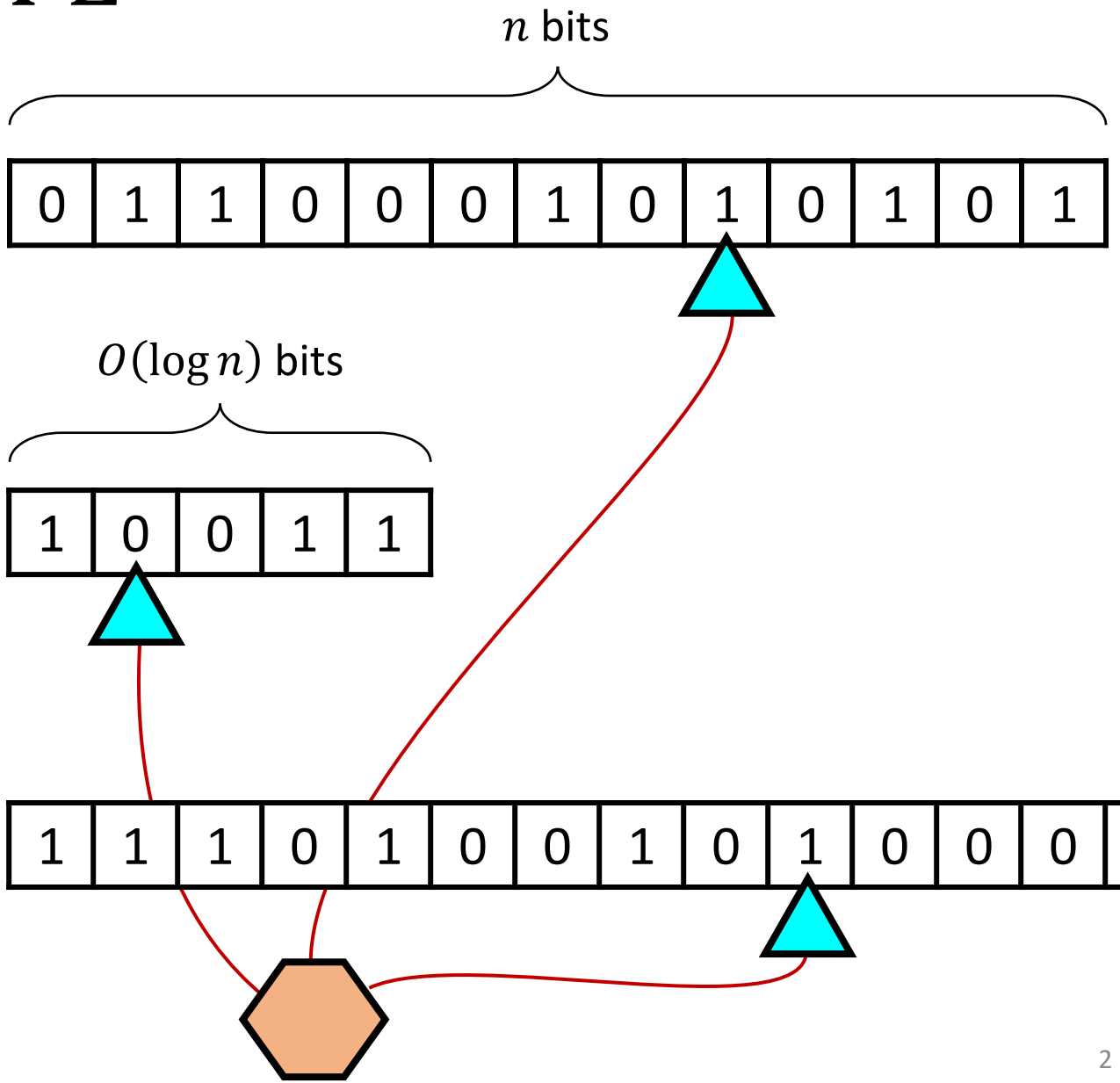
Read-only input tape



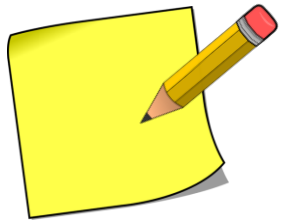
Read-write work tape



Read-once random tape



The complexity class BPL


- Let $f: \{0,1\}^* \to \{0,1\}$
- **Definition:** $f \in$ BPL if there exists a Turing machine M such that:
  - Read-only tape has n-bit input
  - Read/write work tape of size $O(\log n)$
  - Read-once tape filled with random bits
  - $\Pr[M(x) = f(x)] \geq 2/3$
  - M halts for every input and every setting of the random tape

Exercise: Show that M halts in polynomial time

Derandomization of BPL



- Trivially, $\mathrm{L} \subseteq \mathrm{BPL}$
- $\mathrm{L} \overset{?}{=} \mathrm{BPL}$ question intensely studied for several decades

Theorem [OpenAI 2026]: $\mathrm{L} = \mathrm{BPL}$

- "Randomness is not necessary for space-efficient computation"

Disclaimers

- I don’t fully understand the proof!
- The proof is long and complicated
- **Open problem:** Simplify the proof
- I’ll sketch a weaker $\tilde{O}(\log n)$ space proof…
- …which I *also* don’t fully understand!

Disclaimers

- My (incomplete) understanding of the proof is mostly based on discussions with ChatGPT 🤞
- Thanks also to many wonderful humans for sharing insights
- Mistakes are my own
- Some false things I'll say are deliberate simplifications

The concrete problem we will solve

- **Given:** N-vertex directed acyclic graph G, start vertex s, accept vertex t
- **Goal:** Estimate the probability that a random walk from s visits t, to within $\pm \frac{1}{N}$

> **Theorem** [OpenAI 2026]: There is a deterministic algorithm that solves this problem using $O(\log N)$ bits of space

- **Exercise:** The theorem implies $\mathrm{L} = \mathrm{BPL}$

Laplacian approach

"Inverse Laplacian perspective" used by many works on L vs. BPL, starting with [Murtagh, Reingold, Sidford, and Vadhan, FOCS 2017]

- Let $C_0 \in [0,1]^{N \times N}$ be the random walk matrix of G
  - $C_0(x, y) =$ probability of going from x to y in one step
- **Goal:** Approximate the matrix $C_0^0 + C_0^1 + \cdots + C_0^N = (I - C_0)^{-1}$
- **Strategy (v1):** Compute matrices $E_1, E_2, \ldots, E_L$ where $L = O(\log N)$ such that

$$(I + E_L) \cdots (I + E_2) \cdot (I + E_1) \cdot (I - C_0) \approx I$$

- Then $(I - C_0)^{-1} \approx (I + E_L) \cdot (I + E_{L-1}) \cdots (I + E_1)$

Interpretation: Shortcutting

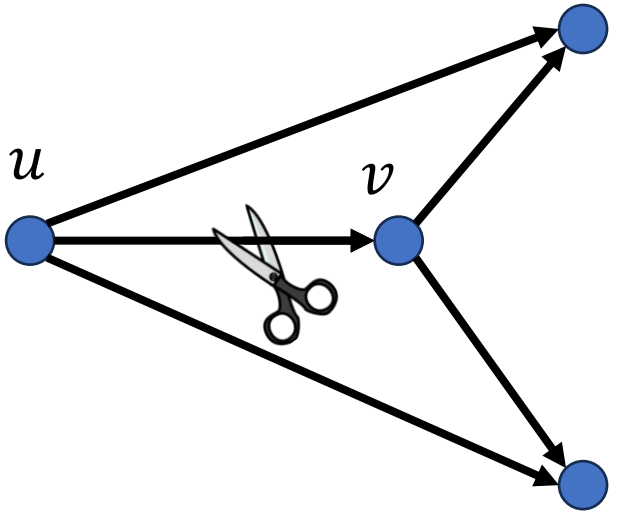
But it will re-appear going from u to out-neighbors of v

$$(I + E) \cdot (I - C) = I - C' \text{ where } C' = C - E + EC$$

We can remove weight from an edge (u, v)…



Photo by Wolfgang Sauber. License: CC-BY-SA 3.0

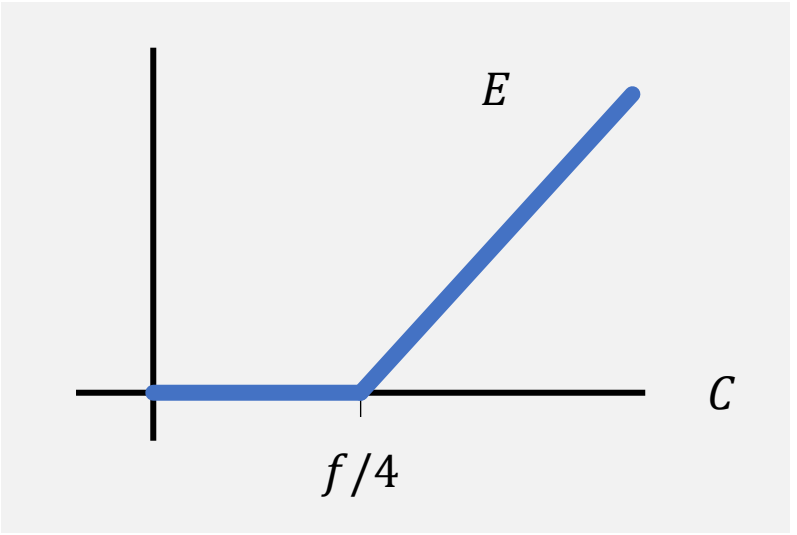


Recap of strategy

- **Goal**: Construct matrices $E_1, E_2, \ldots, E_L$ such that:
  - $L = O(\log N)$
  - $C_L \approx 0$, where $C_i = C_{i-1} - E_i + E_i C_{i-1}$
  - We can approximately compute E_i using $O(\log N)$ bits of space

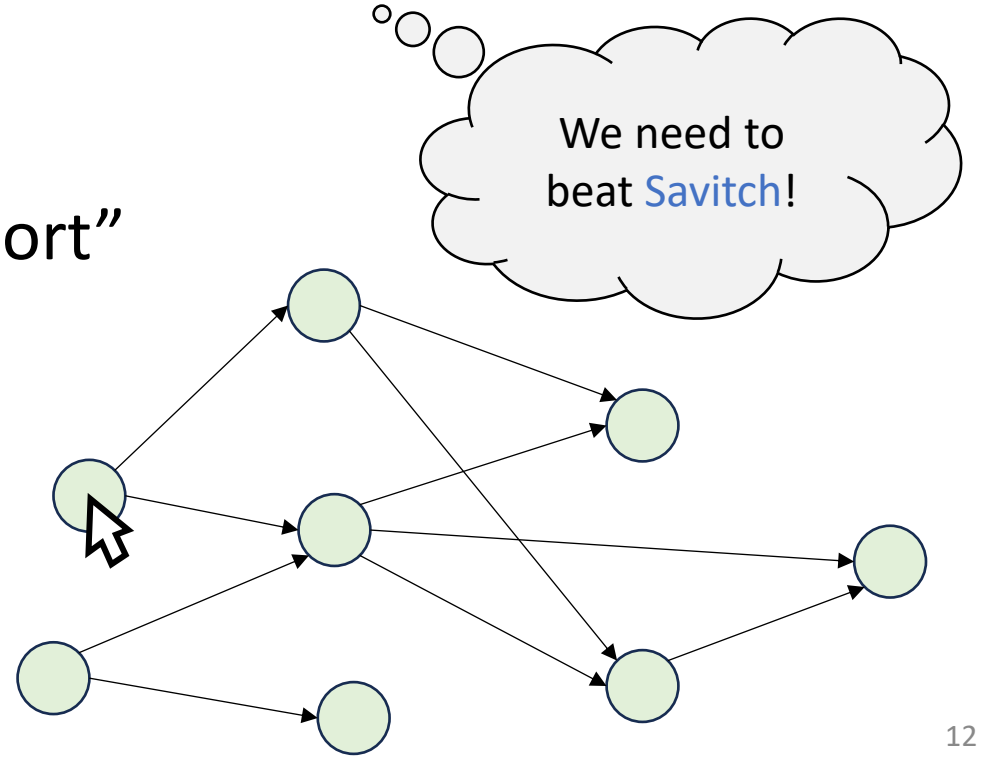
Progress measure: Maximum edge weight

- Let $f = \|C\|_{\max}$
- **Attempt 1**: Let $E(x,y) = (C(x,y) - f/4)_+$
  - Notation: $e_+ \coloneqq \max(0,e) = \mathrm{ReLU}(e)$
- **Attempt 2:** $E \leftarrow E + (C - E + EC - f/4)_+$
- **Lemma:** After $O(1)$ iterations, we get E such that $\|C - E + EC\|_{\max} \leq f/2$
- (Proof omitted)
- So $\|C_L\|_{\max} \leq 1/\mathrm{poly}(n)$ after $L = O(\log N)$ iterations ✅



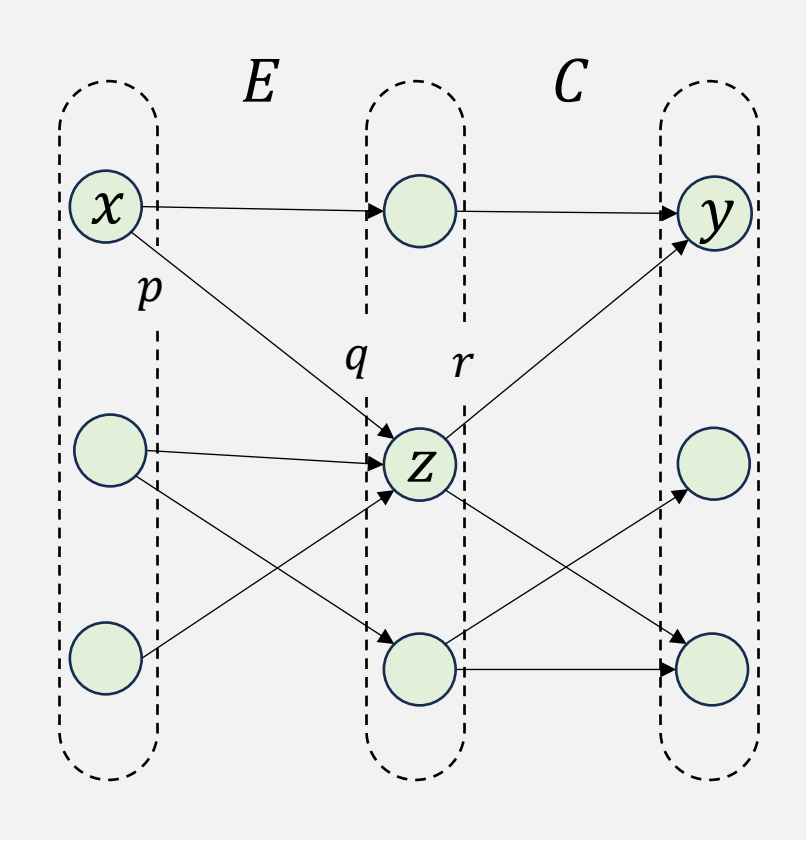
Main challenge: Space complexity

- **Goal:** Approximately compute E_i using only $O(\log N)$ bits of space
- At each moment, just one vertex stored in memory across all recursion levels!
  - Global "cursor"
- Describe an edge using one endpoint and a "port"
  - "p-th outgoing / incoming edge at x"
  - Think: Adjacency list



Space complexity

- $C' = C - E + EC$
- Say we are trying to compute $E(x,z) \cdot C(z,y)$



Time

Cursor	Additional memory			
x	p	r		
x	p	r	$E(x,z)$	
z	q	r	$E(x,z)$	
z	q	r	$E(x,z)$	$C(z,y)$
x	p	r	$E(x,z)$	$C(z,y)$

Wish:

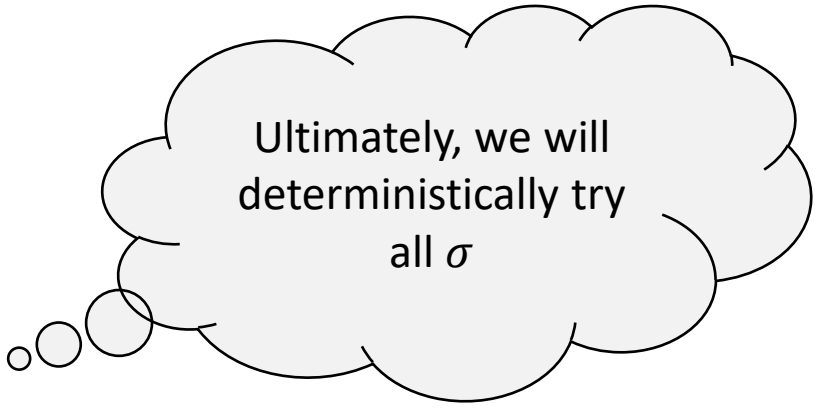
- E has degree $O(1)$
- C has out-degree $O(1)$

Example: Out-degree of E

- $f = \|C\|_{\max}$
- $E(x,y) = (C(x,y) - f/4)_+$ *[ignoring $O(1)$ rounds of iteration]*
- **Wish:** $\deg^+(E) = O(1)$
- **Reality:** $\deg^+(E) = O(1/f)$ and might be $\Theta(1/f)$

Sparsification

Sparsification



- Let $\sigma \in \{0,1\}^{O(\log n)}$ be a random seed
- For each $u \in [0,1]$, sample a set of vertices T_u such that *(simplified)*
  - $\Pr[x \in T_u] = u$
  - Pairwise independence: $\Pr[x \in T_u \text{ and } y \in T_u] = u^2$
  - Nesting: $u \leq t \Rightarrow T_u \subseteq T_t$
- Let $\{U_u\}, \{V_u\}$ be independent copies of $\{T_u\}$
- **Idea:** Only keep edges in, e.g., $T_f \times U_f$

"channel" in OpenAI manuscript

Sparsification

- **Definition (v1):** For $u, t \in [0,1]$ and distinct $X, Y \in \{T, U, V\}$, let

$$C^{X_u, Y_t}(x, y; \sigma) = C(x, y) \cdot 1[x \in X_u] \cdot 1[y \in Y_t]$$

$$E^{X_u, Y_t}(x, y; \sigma) = E(x, y) \cdot 1[x \in X_u] \cdot 1[y \in Y_t]$$

- For now, let's optimistically assume C^{X_f, Y_f} and E^{X_f, Y_f} have degree $O(1)$
  - Not literally true – we will revisit this optimistic assumption later

Computing the sparsified matrices

- Assume we can compute $E^{X,Y}$ and $C^{X,Y}$
- How do we compute $(C')^{X,Y}$?
- Recall $C' = C - E + EC$
- **Note:** $(EC)^{X,Y} \neq E^{X,Y} C^{X,Y}$

Approximating the sparsified graphs

- **Induction hypothesis:** $\forall\, u, t \le f$, $\forall$ distinct $X, Y \in \{T, U, V\}$, can compute $\hat{C}^{X_u, Y_t}, \hat{E}^{X_u, Y_t}$ such that whp over σ, we have

"estimated tables" in OpenAI manuscript

$$\begin{cases} \hat{C}^{X_u, Y_t}(x, y; \sigma) \approx C^{X_u, Y_t}(x, y; \sigma) \\ \hat{E}^{X_u, Y_t}(x, y; \sigma) \approx E^{X_u, Y_t}(x, y; \sigma) \end{cases}$$

"true tables" in OpenAI manuscript

- Can achieve error 2^{-m} using:
  - $O(m + \log(1/u))$ bits of work space

$+O(\log n)$ bits of scratch space, erased before each recursive call

Also, we're ignoring $\log\log N$ terms

  - Global vertex register x ("cursor") and seed register σ, both treated as catalytic memory

Approximating $(EC)^{X,Y}$

- $C' = C - E + EC$
- Focus on EC
- **Goal:** Approximate $(EC)^{X_u,Y_t}(x, y; \sigma) \coloneqq (EC)(x, y) \cdot 1[x \in X_u, y \in Y_t]$
- **Step 1:** Let $\widetilde{EC}^{X_u,Y_t}(x, y; \sigma) = \frac{1}{f}\sum_{z} \hat{E}^{X_u,Z_f}(x, z; \sigma) \cdot \hat{C}^{Z_f,Y_t}(z, y; \sigma)$

"raw sample term" in OpenAI manuscript

$$\widetilde{EC}^{X_u,Y_t}(x,y;\sigma) = \frac{1}{f}\sum_z \hat{E}^{X_u,Z_f}(x,z;\sigma)\cdot\hat{C}^{Z_f,Y_t}(z,y;\sigma)$$

$$\overset{\text{whp}}{\approx} \frac{1}{f}\sum_z E(x,z)\cdot C(z,y)\cdot 1[x\in X_u]\cdot 1[y\in Y_t]\cdot 1\left[z\in Z_f\right]$$

- Therefore, $\mathbb{E}_\sigma\left[\,\widetilde{EC}^{X_u,Y_t}(x,y;\sigma)\mid x\in X_u, y\in Y_t\,\right]\approx (EC)(x,y)$ ✅
- **Remaining challenge:** We need approximation whp, not just in expectation

Approximation whp

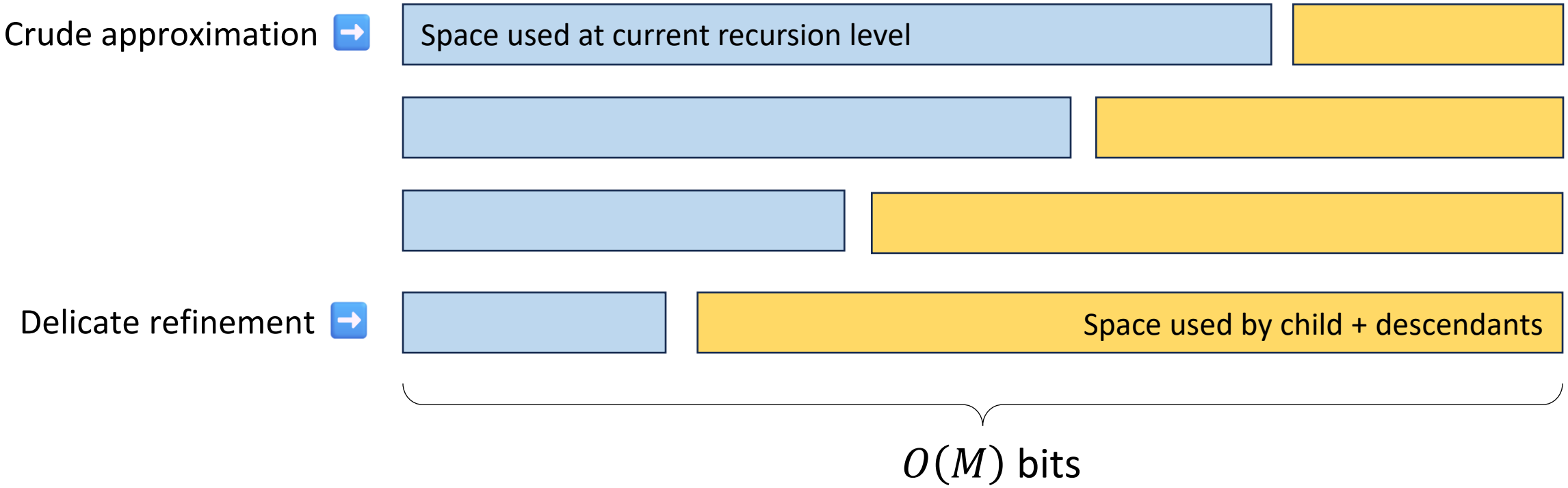
- Let $\Omega = \{\sigma : x \in X_u, y \in Y_t\}$
- **Attempt 1:** Let $\widehat{EC}^{X_u,Y_t}(x,y,\sigma) = 1[\sigma \in \Omega] \cdot \frac{1}{|\Omega|} \sum_{\tau \in \Omega} \left[\widetilde{EC}^{X_u,Y_t}(x,y,\tau)\right]$
- **Issue:** Summing over Ω takes too much space
- We only have one global seed register!



Iterative refinement

Similar spirit as:
- Richardson iteration, e.g., [Murtagh, Reingold, Sidford, Vadhan 2017]
- WPRGs, e.g., [Braverman, Cohen, Garg 2018]

- **Plan:** Start with a crude approximation and iteratively refine until error is 2^{-M}



Expander notation

- Let H be a constant-degree expander on $\Omega = \{\sigma : x \in X_u, y \in Y_t\}$
- For a function $F(x, y; \sigma)$, define

$$(HF)(x, y; \sigma) = \frac{1}{\deg(H)} \sum_{(\sigma,\tau)\in E_H} F(x, y; \tau)$$

Recursion-friendly sampling

Approximation of $C^{Z,Y}$ with error 2^{-m}

- Let $\widetilde{EC}^{X,Y,m}(x,y;\sigma) = \frac{1}{f}\sum_z \hat{E}^{X,Z,m}(x,z;\sigma)\cdot\hat{C}^{Z,Y,m}(z,y;\sigma)$
- Let $\widehat{EC}^{X,Y,M} = \sum_{m=1}^{M} H^{M-m}\left(\widetilde{EC}^{X,Y,m} - \widetilde{EC}^{X,Y,m-1}\right)$ where $\widetilde{EC}^{X,Y,0} \equiv 0$

Lemma: Let $F, F_1, \ldots, F_M : \Omega \to [0,1]$. Assume $\|F - F_m\|_2 \le 2^{-m}$.

Then $\left\|\mathbb{E}[F] - \sum_{m=1}^{M} H^{M-m}(F_m - F_{m-1})\right\|_2 \le O(2^{-M})$.

- **Proof of lemma:** Exercise

Recursion-friendly sampling

Approximation of $C^{Z,Y}$ with error 2^{-m}

- Let $\widetilde{EC}^{X,Y,m}(x,y;\sigma) = \frac{1}{f}\sum_z \hat{E}^{X,Z,m}(x,z;\sigma) \cdot \hat{C}^{Z,Y,m}(z,y;\sigma)$
- Let $\widehat{EC}^{X,Y,M} = \sum_{m=1}^{M} H^{M-m}\left(\widetilde{EC}^{X,Y,m} - \widetilde{EC}^{X,Y,m-1}\right)$ where $\widetilde{EC}^{X,Y,0} \equiv 0$
- **Efficiency:** Heuristically, space complexity of term m should be

$$\underbrace{O(M-m)}_{\text{Expander walk}} + \underbrace{O(m)}_{\text{Recursion}} = O(M)$$ 🥹

- But we are not done yet…

Adjacency matrix vs. adjacency list

$$\widehat{EC}^{X,Y,M} = \sum_{m=1}^{M} H^{M-m} \left(\widetilde{EC}^{X,Y,m} - \widetilde{EC}^{X,Y,m-1} \right)$$

- **So far:** Adjacency matrix of $\widehat{C'}^{X,Y}$ in terms of adjacency matrices of $\hat{C}^{X,Y}$, $\hat{E}^{X,Y}$
- **Need:** Adjacency list of $\widehat{C'}^{X,Y}$ in terms of adjacency lists of $\hat{C}^{X,Y}$, $\hat{E}^{X,Y}$!
- How do we compute information about the p-th outgoing edge of x?
- p-th outgoing edges in C and C' may lead to different destinations!

Adjacency lists

A simplifying assumption

- Recall $f = \|C\|_{\max}$
- For simplicity, assume every nonzero entry of C is $\Theta(f)$
  - Also assume the same about C'
- (Actual proof: Divide into bins, treat each bin separately, etc.)

Updates

- **So far:** We have constructed a good approximation of $(C')^{X,Y}$ of the form

$$\sum_{m=1}^{M-1} H^{M-m} \cdot \left(\tilde{F}_m - \tilde{F}_{m-1}\right) = \sum_{j=1}^{\log M} \Delta_j$$ ← "Update j"

- $\tilde{F}_m$ combines $\hat{C}^{X,Y,m}$, $\hat{E}^{X,Y,m}$, $\hat{C}^{Z,Y,m}$, $\hat{E}^{X,Z,m}$
- $\Delta_1 =$ first $M/2$ terms, $\Delta_2 =$ next $M/4$ terms, $\Delta_3 =$ next $M/8$ terms, etc.
- We have $\deg^+(\Delta_j) = 2^{O(S_j)}$ where $S_j = M/2^j$

Compression

- $(C')^{X,Y} \approx \sum_{j=1}^{\log M} \Delta_j$
- Let $\widehat{C'}^{X,Y,M} = T_{\log M}$ where $T_0 = 0$ and $T_j = \text{Compress}_j\left(T_{j-1} + \Delta_j\right)$
  - $\text{Compress}_j(T)$ deletes edges with absolute weight less than $f \cdot 2^{-cS_j}$ where $S_j = M/2^j$
- **Interpretation:** Better and better approximations of $(C')^{X,Y}$
- Getting sparser: $\deg^+\left(T_j\right) = 2^{O\left(S_j\right)}$ culminating in $\deg^+\left(T_{\log M}\right) = O(1)$

Correctness

$$\widehat{C'}^{X,Y,M} = T_{\log M} \text{ where } T_0 = 0 \text{ and } T_j = \text{Compress}_j\left(T_{j-1} + \Delta_j\right)$$

- **Claim:** Compress_j doesn't introduce much approximation error
- **Intuition:**
  - Case 1: j is small (crude approximation). Then Compress_j only removes edges that have near-zero weight ✅
  - Case 2: j is large (delicate refinement). By simplifying assumption, every nonzero edge weight in $(C')^{X,Y}$ is $\Theta(f)$, so Compress_j only removes erroneous edges ✅

Efficiency

$$\widehat{C'}^{X,Y,M} = T_{\log M} \text{ where } T_0 = 0 \text{ and } T_j = \text{Compress}_j\left(T_{j-1} + \Delta_j\right)$$

Lemma: Can navigate T_j using $O\left(S_j\right)$ bits of space, plus queries to T_{j-1} and $\tilde{F}_m$ where $m = M - \Theta\left(S_j\right)$, plus global vertex/seed registers, plus log log terms that we ignore, plus $O(\log n)$ bits of scratch space that is erased before any queries

- As we will see, the proof of the lemma isn't hard – just unpack definitions

Merging edges

- $T_j = \text{Compress}_j(T_{j-1} + \Delta_j)$
- Let's be a little more precise about the notation:
  - $T_{j-1} + \Delta_j$ denotes multi-graph constructed by interpreting $+$ as union of edge sets
  - $\text{Merge}(T_{j-1} + \Delta_j)$ denotes graph in which we merge edges with same endpoints
  - $\text{Compress}_j(T_{j-1} + \Delta_j)$ applies Merge and then deletes light-weight edges

Navigating T_j

- $T_j = \text{Compress}_j(T_{j-1} + \Delta_j)$
- To compute information about the p-th outgoing edge of x in T_j:
  1. Initialize $r \leftarrow 0$
  2. For $q \in \left[2^{O(S_j)}\right]$:
     a) If absolute weight of q-th outgoing edge of x in $\text{Merge}(T_{j-1} + \Delta_j)$ is $\geq f \cdot 2^{-cS_j}$, increment r
     b) If $r = p$, then we've found the edge we're interested in
- Space $= O(S_j)$ + space complexity of navigating $\text{Merge}(T_{j-1} + \Delta_j)$

Navigating $\mathrm{Merge}(T_{j-1} + \Delta_j)$

- Let Q be the set of q such that p-th and q-th outgoing edges of x in $T_{j-1} + \Delta_j$ have the same destination
- If $p \neq \min(Q)$, then p-th outgoing edge of x in $\mathrm{Merge}(T_{j-1} + \Delta_j)$ is `null`
- If $p = \min(Q)$, then weight of edge in $\mathrm{Merge}(T_{j-1} + \Delta_j)$ is sum of weights of Q-edges in $T_{j-1} + \Delta_j$
- Space $= O(S_j)$ + space complexity of navigating $T_{j-1} + \Delta_j$, including comparing destinations

Comparing destinations

- To figure out whether p-th and q-th outgoing edges of x lead to the same destination in $T_{j-1} + \Delta_j$:
  1. For $k \in [\log N]$:
     a) Compute k-th bit of name of destination of p-th outgoing edge of x in $T_{j-1} + \Delta_j$
     b) Compute k-th bit of name of destination of q-th outgoing edge of x in $T_{j-1} + \Delta_j$
     c) If they don't match, output "No"
  2. Output "Yes"
- Space $= O(\log \log N) +$ space complexity of computing destinations in $T_{j-1} + \Delta_j$

Navigating $T_{j-1} + \Delta_j$

- Δ_j is a sum of $M/2^j$ terms of the form $H^{M-m} \cdot \left(\tilde{F}_m - \tilde{F}_{m-1}\right)$
- $m = M - \Theta\left(S_j\right)$ and $\tilde{F}_m$ combines $\hat{C}^{X,Y,m}$, $\hat{E}^{X,Y,m}$, $\hat{C}^{Z,Y,m}$, $\hat{E}^{X,Z,m}$
- $O\left(S_j\right)$ bits of space used, plus queries to T_{j-1} and $\tilde{F}_m$
- That completes the proof of:

Lemma: Can navigate T_j using $O\left(S_j\right)$ bits of space, plus queries to T_{j-1} and $\tilde{F}_m$ where $m = M - \Theta\left(S_j\right)$, plus unimportant terms

Adding up space used for all T_j

Lemma: Can navigate T_j using $O(S_j)$ bits of space, plus queries to T_{j-1} and $\tilde{F}_m$ where $m = M - \Theta(S_j)$, plus unimportant terms

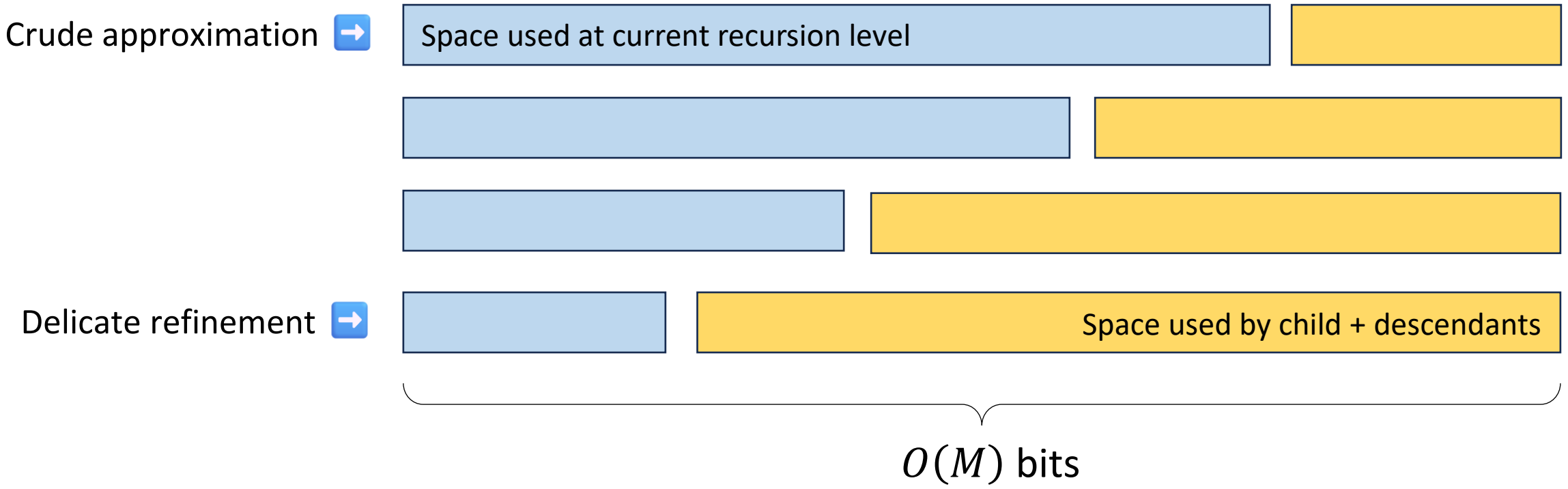
Query	Space used
$T_{\log M}$	$c_1 \cdot 1$
$T_{\log M-1}$	$c_1 \cdot 2$
$\vdots$	$\vdots$
T_j	$c_1 \cdot M/2^j$
$\tilde{F}_m$	$c_2 \cdot (M - M/2^j)$

$$\text{Total} = c_1 \cdot (1 + 2 + 4 + \cdots + M/2^j) + c_2 \cdot (M - M/2^j)$$

$$\leq 2c_1 \cdot M/2^j + c_2 \cdot (M - M/2^j)$$

$$\leq c_2 \cdot M, \text{provided we choose } c_2 \geq 2c_1$$

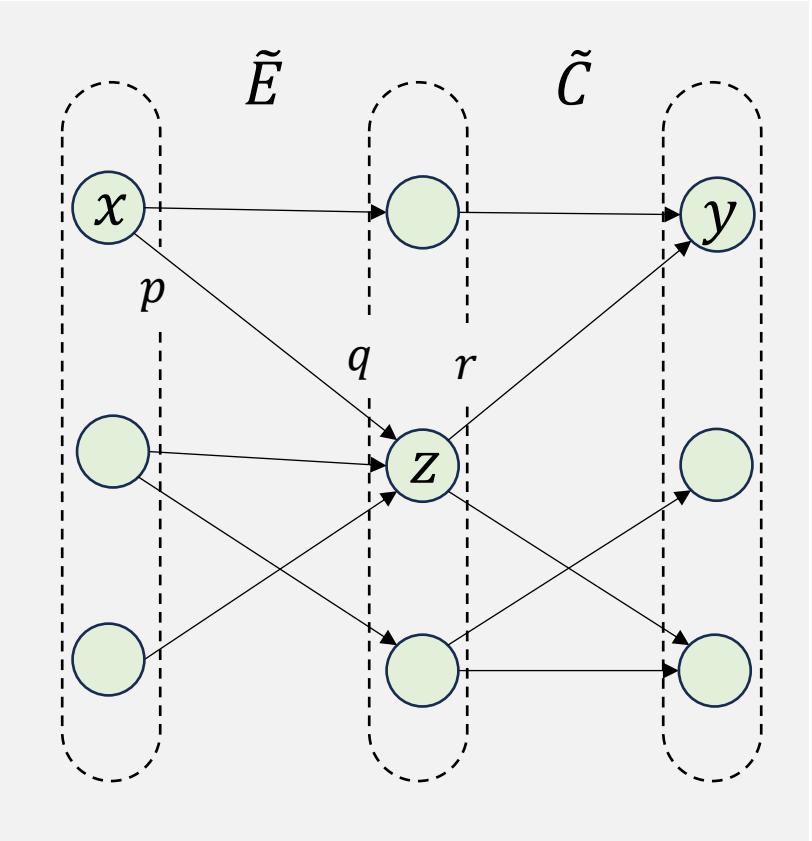
Iterative refinement



Incoming ports and two-way navigation

Two-way navigation

- Recall strategy for computing $\tilde{E}(x,z) \cdot \tilde{C}(z,y)$



Time

Cursor	Additional memory			
x	p	r		
x	p	r	$\tilde{E}(x,z)$	
z	q	r	$\tilde{E}(x,z)$	
z	q	r	$\tilde{E}(x,z)$	$\tilde{C}(z,y)$
x	p	r	$\tilde{E}(x,z)$	$\tilde{C}(z,y)$

- **So far:** How to compute $(x,p) \mapsto z$. **Need to compute:** $(x,p) \leftrightarrow (z,q)$

One-way navigator ⇒ two-way navigator

- Generic "upgrade" theorem based on classic work on space-bounded computation [Sipser 1980] [Cook, McKenzie 1987] [Lange, McKenzie, Tapp 2000]
- Roughly: Whenever one can efficiently compute the map $(x, p) \mapsto z$, one can also efficiently compute the maps $(x, p) \leftrightarrow (z, q)$
- Let's prove a clean, simplified version of the theorem

Numberings and rotation maps

These concepts are used in prior work on L vs. BPL, e.g., [Reingold 2008]

- Let A be a simple D-regular directed graph on vertex set V



- A one-way numbering of A is a function $\Gamma: V \times [D] \to V$ such that for every $x \in V$, the vertices $\Gamma(x, 1), \ldots, \Gamma(x, D)$ are the D out-neighbors of x
- A two-way numbering of A is a pair (Γ, Γ'), where Γ is a one-way numbering of A and Γ' is a one-way numbering of "reverse" graph $A^{\top}$
  - If $\Gamma(x, p) = y$ and $\Gamma'(y, p') = x$, we write $\mathrm{Rot}(x, p) = (y, p')$
  - Rotation map $\mathrm{Rot}: V \times [D] \to V \times [D]$



One-way navigator ⇒ two-way navigator

[Sipser 1980] [Cook, McKenzie 1987] [Lange, McKenzie, Tapp 2000]

- Assume A has a one-way numbering $\Gamma: V \times [D] \to V$ that can be computed in-place using s bits of scratch space: $(x, p, 0^s) \mapsto (y, 1, 0^s)$

Theorem: There is a two-way numbering such that the rotation map $\mathrm{Rot}: V \times [D] \to V \times [D]$ and its inverse can both be computed in-place using $s' = s + O(\log D)$ bits of scratch space: $(x, p, 0^{s'}) \leftrightarrow (y, q, 0^{s'})$.

Proof sketch

- Consider the computation graph $\mathcal{M}$ of the computation of Γ
- Configurations from which $(y, 1, 0^s)$ is reachable form a tree
- Explore tree in "depth-first search" order from $(x, p, 0^s)$ until we traverse last incoming edge of $(y, 1, 0^s)$
- Output (y, q), where q is the number of $(x', p', 0^s)$ configurations visited
- Space overhead: $O(\log D)$ bits for counter q, plus $O(1)$ bits for ports in $\mathcal{M}$

Some difficulties we have glossed over

Ensuring that sparsified graphs are sparse

- **Definition (v1):** For $u, t \in [0,1]$ and distinct $X, Y \in \{T, U, V\}$, let

$$C^{X_u, Y_t}(x, y; \sigma) = C(x, y) \cdot 1[x \in X_u] \cdot 1[y \in Y_t]$$

$$E^{X_u, Y_t}(x, y; \sigma) = E(x, y) \cdot 1[x \in X_u] \cdot 1[y \in Y_t]$$

- So far, we've optimistically assumed C^{X_f, Y_f} and E^{X_f, Y_f} have degree $O(1)$

- Now it is time to face reality

Out-degree of E

- Recall: $E(x,y) = (C(x,y) - f/4)_+$

  [ignoring $O(1)$ rounds of iteration]

- $E(x,y) > 0 \Rightarrow C(x,y) > f/4$

- $\deg^+(E) \leq O(1/f)$

- A vertex in E^{X_f, Y_f} typically has $O(1)$ out-neighbors, but not always 😟

🌠 Wish:

- E has out-degree $O(1)$
- E has in-degree $O(1)$
- C has out-degree $O(1)$

“Row gate”



- Say that x is “good” if $|\{y \in Y_t : E(x,y) > 0\}| \leq 1000$
- Let $q_{x,y}^{X_u,Y_t} = \Pr[\, x \text{ is good} \mid x \in X_u, y \in Y_t \,] \geq 0.99$
- **Definition (v2):** $E^{X_u,Y_t}(x,y;\sigma) = E(x,y) \cdot 1[x \in X_u] \cdot 1[y \in Y_t] \cdot \frac{1[x \text{ is good}]}{q_{x,y}^{X_u,Y_t}}$
- This way, for every σ, we have $\deg^+(E^{X_u,Y_t}) \leq 1000$ ✅

Out-degree of C



🌠 Wish:

- E has out-degree $O(1)$
- E has in-degree $O(1)$
- C has out-degree $O(1)$

- Recall simplifying assumption: Every nonzero edge weight in C is $\Theta(f)$
- Therefore, $\deg^+(C) \leq O(1/f)$
- Use row gate to guarantee $\deg^+(C^{X_u,Y_t}) = O(1)$

In-degree of E

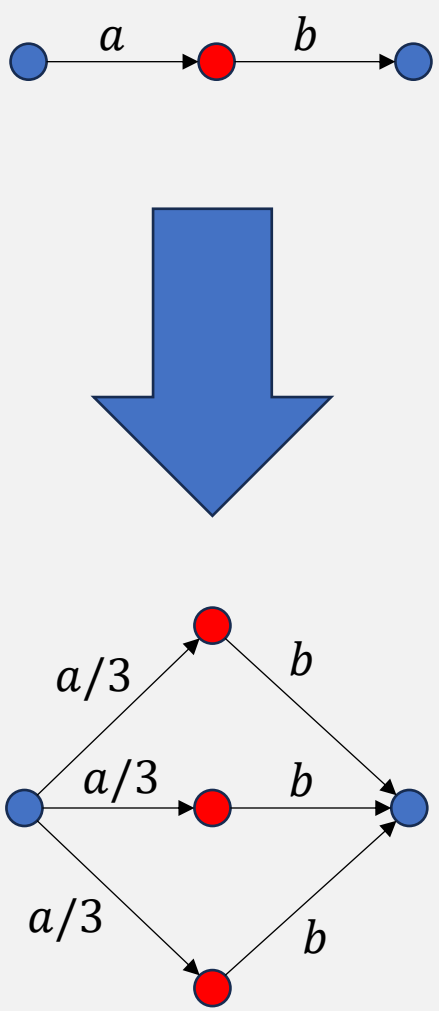
🌠 Wish:

- E has out-degree $O(1)$
- E has in-degree $O(1)$
- C has out-degree $O(1)$

- $\deg^+(E) \leq O(1/f)$
- So the average in-degree is $O(1/f)$…
- …but there are some “exceptional vertices” with high in-degree 😟

Solution: Splitting vertices

- Vertex y is "good" if $|\{x: C(x,y) > f/4\}| \leq 1000/f$
- **Definition (v2)**: $E(x,y) = (C(x,y) - f/4)_+ \cdot 1[y \text{ is good}]$
- **Definition (v2)**: Construct C' from $C - E + EC$ by splitting each bad vertex into a "cloud" of $O(1)$ vertices
- $\|C\|_{\max}$ still decreases from f down to $f/2$, and # vertices only increases by a factor of 1.01



Shortcutting and splitting: Two distinct weapons



Photo by Wolfgang Sauber. License: CC-BY-SA 3.0

Linear algebra of vertex splitting

- **Goal:** Compute $C_0^0 + C_0^1 + \cdots + C_0^N = (I - C_0)^{-1}$
- **Strategy (v2):** $(I - C_0)^{-1} \approx A \cdot P_L \cdot (I + E_L) \cdots P_1 \cdot (I + E_1)$ where P_i splits bad vertices into clouds and A deletes all but one member of each cloud
- A and P_i are rectangular matrices

In-degree of E



🌠 Wish:

- E has out-degree $O(1)$
- E has in-degree $O(1)$
- C has out-degree $O(1)$

- Under new definition, $\deg^-(E) \leq O(1/f)$
- Each vertex in E^{X_u,Y_t} has expected in-degree $O(1)$
- Use a "column gate" to guarantee $\deg^-(E^{X_u,Y_t}) = O(1)$

Another issue: Hard thresholding

- We have used hard thresholds in many places
- E.g., split vertices if # substantial incoming edges is more than 1000
- **Difficulty:** This rule is sensitive to tiny approximation errors
- **Solution:** Smooth out all the threshold rules
- E.g., always split vertices, but use a smooth rule to distribute incoming edge weight among copies

Another issue: Conditional expander walk

- We do an expander walk on the vertex set $\Omega = \{\sigma : x \in X_u, y \in Y_t\}$
- But we don't know y! How can we compute neighbors in Ω?
- Solution involves constructing expander and vertex sampler with special algebraic properties
- I don't understand this part yet
- Proof of expansion uses "property (T)"

Conclusions

- Proof is complicated, but not beyond human comprehension
- Flavor is more “algorithm design” than “math”
  - (Or maybe it just seems that way to me because I don’t understand the more “mathy” parts yet…)
- **More general theorem:** For every $S = S(n) \geq \log n$, we have

$$\mathrm{BPSPACE}\big(O(S)\big) = \mathrm{DSPACE}\big(O(S)\big)$$

Conclusions

- **Still open:** Design an explicit pseudorandom generator that stretches $O(\log n)$ truly random bits to n pseudorandom bits that can be used to simulate log-space computation
- **Thanks! Questions?**

Just for fun, here are some word frequencies in the OpenAI manuscript, expanding on observations by Jake Ruotolo

